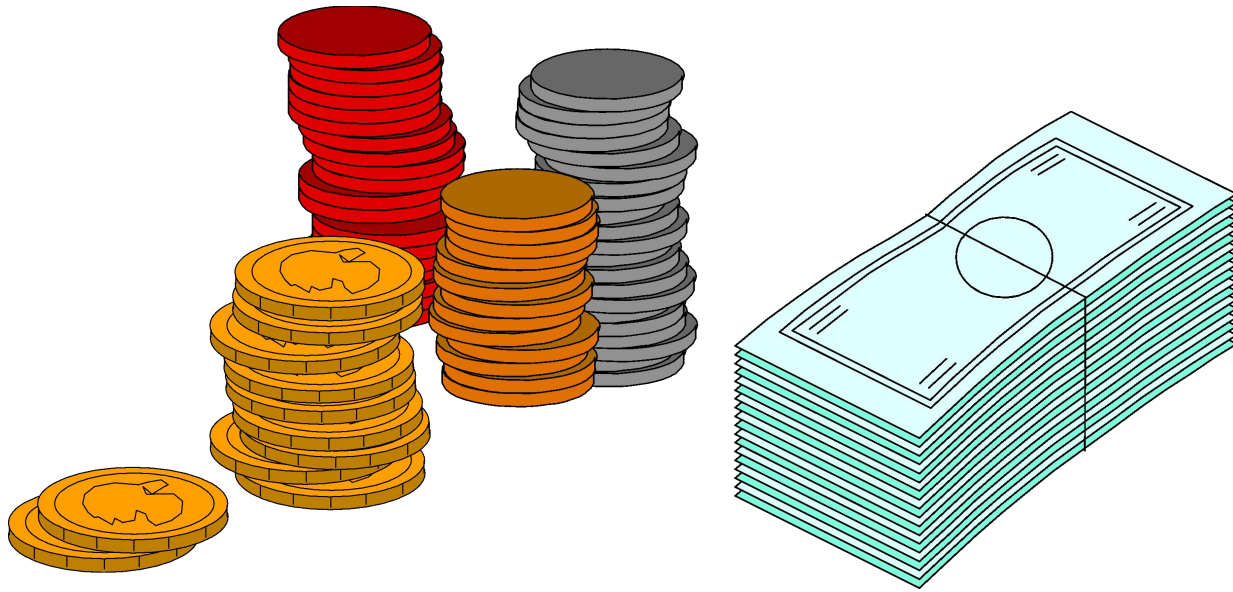


# Stacks

- Introduction
- Nature of the Stack
- An Array-Based Implementation
- Applications
  - Checking for Balanced Braces
  - Recognizing Strings in a Language
  - Evaluating Postfix Expressions
- A Pointer-Based Implementation

# Stacks of Coins and Bills

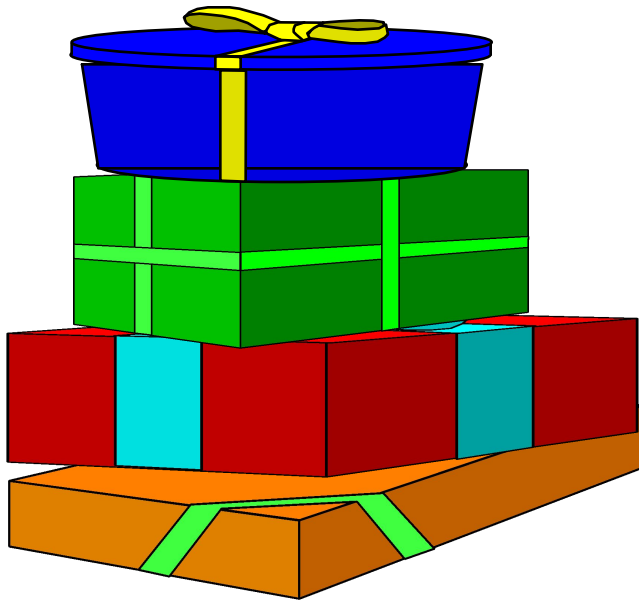


# Nature of the Stack

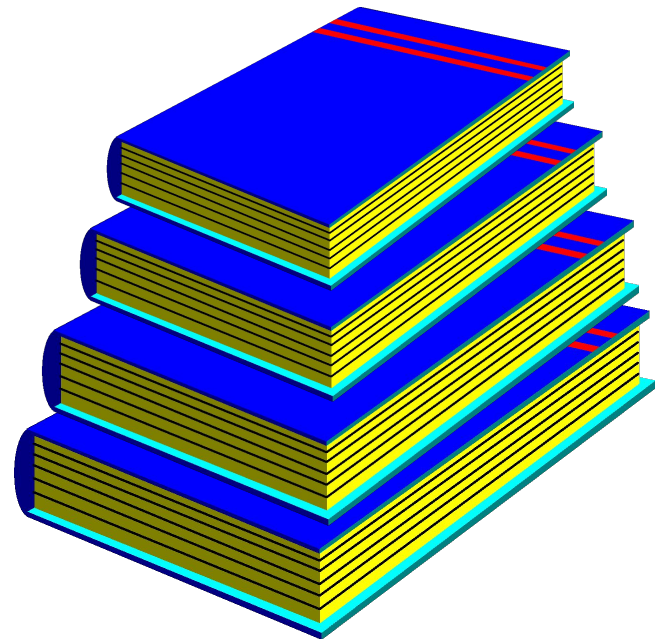
- Logical level: A stack is an ordered group of homogeneous items (elements), in which the removal and addition of stack items can take place only at the top of the stack.
- A stack is a LIFO “last in, first out” structure.

# Stacks of Boxes and Books

**TOP OF THE STACK**



**TOP OF THE STACK**



# Stack Operations

- **createStack( )**

//Creates an empty stack.

- **destroyStack( )**

//Destroys a stack.

- **isEmpty( )**

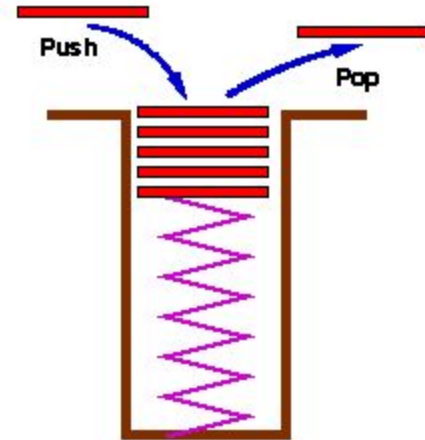
//Determines whether a stack is empty.

- **push (stackItemType NewItem )**

//Adds NewItem to top of a stack.

- **pop ( )**

//Removes the top of a stack the item that was added most recently.



# Stack Operations

- **pop (stackItemType stackTop)**

//Retrieves into stackTop and then removes the top of a stack. That is, retrieves and removes the item that was added most recently.

- **getTop(stackItemType stackTop)**

//Retrieves into stackTop the top of a stack. That is, retrieves the item that was added most recently. Retrieval does not change the stack.

letter

'V'

## Tracing Client Code

Private data:

top



[MAX\_ITEMS-1]

⋮

[ 2 ]

[ 1 ]

items [ 0 ]



```
char letter = 'V';
```

```
StackType charStack;
```

```
charStack.Push(letter);
```

```
charStack.Push('C');
```

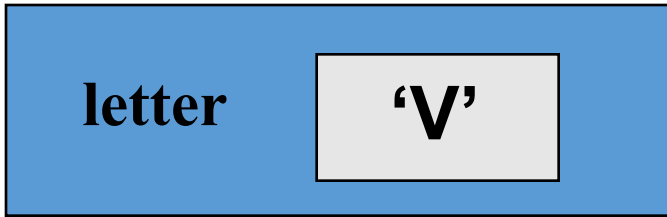
```
charStack.Push('S');
```

```
if ( !charStack.IsEmpty( ))  
    charStack.Pop(letter);
```

```
charStack.Push('K');
```

```
while (!charStack.IsEmpty( ))  
    charStack.Pop(letter);
```

# Tracing Client Code



**Private data:**

top

-1

[MAX\_ITEMS-1]

⋮

[ 2 ]

[ 1 ]

items [ 0 ]



```
char letter = 'V';
```

```
StackType charStack;
```

```
charStack.Push(letter);
```

```
charStack.Push('C');
```

```
charStack.Push('S');
```

```
if ( !charStack.IsEmpty( ))  
    charStack.Pop(letter);
```

```
charStack.Push('K');
```

```
while (!charStack.IsEmpty( ))  
    charStack.Pop(letter);
```

# Tracing Client Code

letter

'V'

Private data:

top

0

[MAX\_ITEMS-1]

⋮

[ 2 ]

[ 1 ]

items [ 0 ]

'V'

```
char letter = 'V';  
StackType charStack;
```

```
charStack.Push(letter);
```

```
charStack.Push('C');
```

```
charStack.Push('S');
```

```
if ( !charStack.IsEmpty( )  
    charStack.Pop(letter);
```

```
charStack.Push('K');
```

```
while (!charStack.IsEmpty( ))  
    charStack.Pop(letter);
```

letter

'V'

## Tracing Client Code

Private data:

top

1

[MAX\_ITEMS-1]

⋮

[ 2 ]

[ 1 ]

items [ 0 ]

'C'

'V'

```
char letter = 'V';  
StackType charStack;  
charStack.Push(letter);
```

```
charStack.Push('C');
```

```
charStack.Push('S');  
if ( !charStack.IsEmpty( ) )  
    charStack.Pop(letter);  
charStack.Push('K');  
while ( !charStack.IsEmpty( ) )  
    charStack.Pop(letter);
```

letter

'V'

## Tracing Client Code

Private data:

top 2

[MAX\_ITEMS-1]

⋮

[ 2 ]

'S'

[ 1 ]

'C'

items [ 0 ]

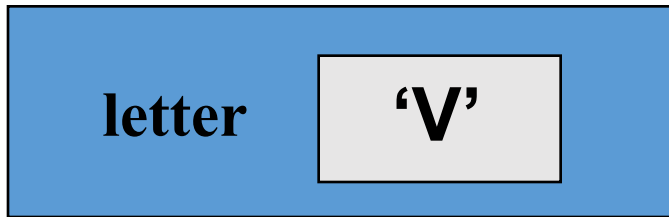
'V'

```
char letter = 'V';  
StackType charStack;  
charStack.Push(letter);  
charStack.Push('C');
```

```
charStack.Push('S');
```

```
if ( !charStack.IsEmpty( ) )  
    charStack.Pop(letter);  
charStack.Push('K');  
while ( !charStack.IsEmpty( ) )  
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**2**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

**'S'**

**[ 1 ]**

**'C'**

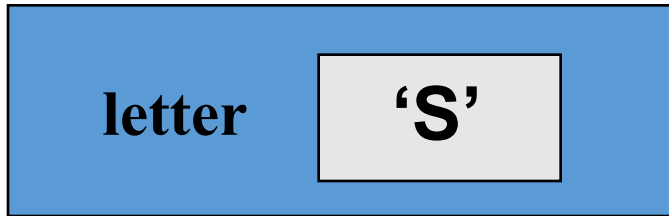
**items [ 0 ]**

**'V'**

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');

if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**1**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

**[ 1 ]**

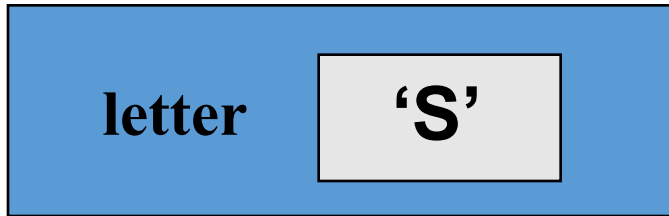
**items [ 0 ]**

**'C'**

**'V'**

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**2**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

**'K'**

**[ 1 ]**

**'C'**

**items [ 0 ]**

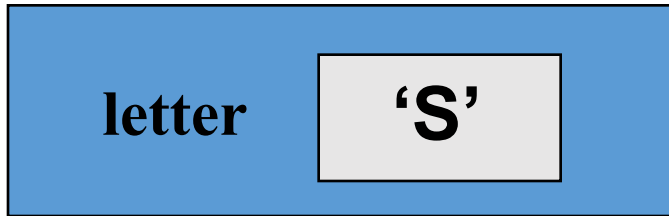
**'V'**

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
```

```
charStack.Push('K');
```

```
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**2**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

**'K'**

**[ 1 ]**

**'C'**

**items [ 0 ]**

**'V'**

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code

letter

'K'

Private data:

top

1

[MAX\_ITEMS-1]

⋮

[ 2 ]

[ 1 ]

items [ 0 ]

'C'

'V'

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**1**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

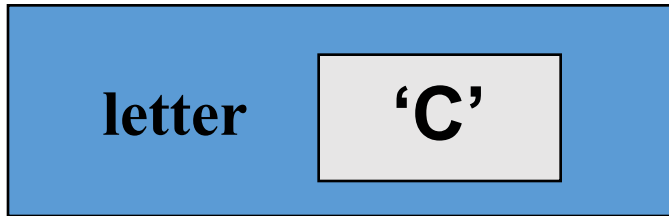
**[ 1 ]**

**items [ 0 ]**



```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**0**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

**[ 1 ]**

**items [ 0 ]**

**'V'**

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code

letter

'C'

Private data:

top

0

[MAX\_ITEMS-1]

:

[ 2 ]

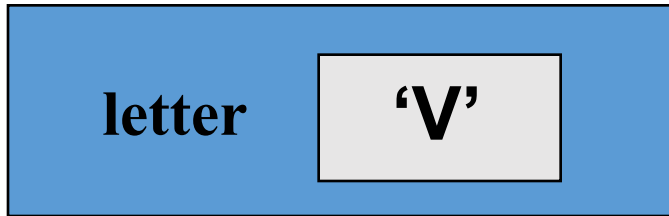
[ 1 ]

items [ 0 ]

'V'

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## Tracing Client Code



**Private data:**

**top**

**-1**

**[MAX\_ITEMS-1]**

**:**  
**:**  
**:**

**[ 2 ]**

**[ 1 ]**

**items [ 0 ]**



```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

## End of Trace

letter

**'V'**

**Private data:**

top

**-1**

[MAX\_ITEMS-1]

:

[ 2 ]

[ 1 ]

items [ 0 ]

```
char letter = 'V';
StackType charStack;
charStack.Push(letter);
charStack.Push('C');
charStack.Push('S');
if ( !charStack.IsEmpty( ))
    charStack.Pop(letter);
charStack.Push('K');
while (!charStack.IsEmpty( ))
    charStack.Pop(letter);
```

# Implementations of the Stack

# Array-based Implementation

```
const int MAX_STACK = maximum-size-of-stack;
```

```
typedef desired-type-of-stack-item StackItemType;
```

```
class Stack
```

```
{
```

```
public:
```

```
// constructors and destructor:
```

```
    Stack(); // default constructor
```

```
    // copy constructor and destructor are supplied by the compiler
```

```
// stack operations:
```

```
    bool isEmpty( ) const;
```

```
    // Determines whether a stack is empty.
```

```
    // Precondition: None
```

```
    // Postcondition: Returns true if the stack is empty; otherwise returns false.
```

# Array-based Implementation

```
void push(StackItemType newItem);
```

```
// Adds an item to the top of a stack.
```

```
// Precondition: newItem is the item to be added.
```

```
// Postcondition: If insertion is successful, newItem is on the top of  
the stack.
```

```
void pop( );
```

```
// Removes the top of a stack.
```

```
// Precondition: None.
```

```
// Postcondition: If the stack is not empty, the item that was added  
most recently is removed. However, if the stack is empty, deletion is  
impossible.
```

# Array-based Implementation

```
void pop( StackItemType& stackTop);  
// Retrieves and removes the top of a stack.  
// Precondition: None.  
// Postcondition: If the stack is not empty, stackTop contains the  
// item that was added most recently and the  
// item is removed. However, if the stack is empty, deletion is  
// impossible and stackTop is unchanged.
```

```
void getTop(StackItemType& stackTop) const;  
// Retrieves the top of a stack.  
// Precondition: None.  
// Postcondition: If the stack is not empty, StackTop  
// contains the item that was added most recently. However,  
// if the stack is empty, the operation fails and stackTop is  
// unchanged. The stack is unchanged.
```

# Array-based Implementation

private:

```
    StackItemType items[MAX_STACK]; // array of stack items
    int          top;                // index to top of stack
}; // end class
// End of header file.
```

# Array-based Implementation

```
Stack::Stack( ): top(-1)
{
} // end default constructor

bool Stack::isEmpty( ) const
{
    return top < 0;
} // end isEmpty

void Stack::push(StackItemType newItem)
{ //if stack has no more room for another item
    if ( top >= MAX_STACK - 1)
        cout <<endl<<"Cannot push item";
    else
    { ++top;
      items[top] = newItem;
    } // end if
} // end push
```

# Array-based Implementation

```
void Stack::pop( )
{
    if (isEmpty ( ) )
        cout<<endl<<"Cannot pop item";
    else
        --top;    // stack is not empty; pop top
} // end pop
```

```
void Stack::pop( StackItemType& stackTop)
{
    if (isEmpty ( ) )
        cout<<endl<<"Cannot pop item";
    else
    { //stack is not empty; retrieve top
        stackTop = items [top];
        --top;    // pop top
    } //end if
} // end pop
```

# Array-based Implementation

```
void Stack::getTop(StackItemType& stackTop) const
{
    if ( isEmpty( ) )
        cout<<endl<<"cannot return item";
    else
        //stack is not empty ; retrieve top
        stackTop = items[top];
} //end getTop
// End of implementation file.

int main( )
{
    StackItemType anItem;
    Stack aStack;

    cin >> anItem;           // read an item
    aStack.Push(anItem);      // push it onto stack
}
```

# Applications of the Stack

## Checking for Balanced braces

- C++ Uses Curly Braces to Delimit Groups of Statements
- Stack can be Used to Check Balanced Braces
- The Braces can be Matched in Order and Number

### Examples

- `abc{defg{ijk}{l{mn}}op}qr`
- `abc{def}}{ghij{kl}m`

### Requirement for Balanced Braces

- Each Time You Encounter an “}”, it Matches an Already Encountered “{”
- When You Reach the End of the String, You have Matched Each “{”

# Checking for Balanced braces

```
aStack.createStack( )
balanced SoFar = TRUE
i = 0
while( balancedSoFar and i < length of aString )
{ch = character at position i in aString
  ++i
  // push an open brace
  if ( ch is '{' )
    aStack.push( '{ ' )
  // close brace
  else if ( ch is '}' )
    if ( !aStack.isEmpty ( ) )
      aStack.pop ( )      //pop a matching open brace
    else                  //no matching open brace
      balancedSoFar = false
```

# Checking for Balanced braces

```
    //ignore all characters other than braces
} //end while

if ( balancedSoFar and aStack.isEmpty ( ) )
    aString has Balanced Braces
else
    aString does not have balanced Braces
```

# Applications of the Stack

## Recognizing Strings in a Language

- Recognizing Whether a Particular String is in the Language
- $L = \{ w \$ w' : w \text{ is a Possibly Empty String of Characters Other Than } \$, w' = \text{reverse}(w) \}$

### Examples

- A \$ A, \$, ABC\$CBA
- AB\$AB, ABC\$CB

### Recognizing Strings in a Language

1. A Stack is very Useful in Determining Whether a Given String is in L.
2. If you Traverse the First Half of the String and Push Each Character Onto a Stack, When you Reach the \$ You can Undo the Process
3. For Each Character in the Second Half of the String, You Pop a Character Off the Stack

# Applications of the Stack

## Recognizing Strings in a Language

### Recognizing Strings in a Language

- You Must Match the Popped Character With the Current Character in the String to Ensure That the Second Half of the String is the Reverse of the First Half
- The Stack Must be Empty When You Reach the End of the String

# Recognizing Strings in a Language

```
aStack.createStack( )
//push the characters before $, that is, the characters in w
i = 0
ch = character at position i in aString
while ( ch is not '$' )
{aStack.push ( ch )
  ++i
  ch = character at position i in aString
} //end while
skip the $
++i
//match the reverse of w
inLanguage = true //assume string is in Language
while ( inLanguage and i < length of aString )
{if ( aStack.isEmpty ( ) )
  //first half of string is shorter than second half
  inLanguage = false           //reject string
```

# Recognizing Strings in a Language

```
else if ( top of stack is not the character at position i in aString)
    //characters do not match
    inLanguage = false      //reject string
else //characters match
{
    aStack.pop ( Success)
    ++i
} //end if
} //end while
if ( inLanguage and aStack.isEmpty( ) )
    aString is in language
else
    aString is not in language
```

# Applications of the Stack

## Evaluating Postfix Expressions

- Some Calculators use a Postfix Convention

- Example

To Compute the Value of  $2*(3+4)$

- Enter the Sequence 2,3,4,+, \*, Corresponding to  $234+*$
- An Operator in Postfix Expression Applies to the Two Operands that Immediately Precede it
- The Calculator Must be Able to Retrieve the Operands Entered Most Recently
- The ADT Stack Provides this Capability

Assumptions

- The String is a Syntactically Correct Postfix Expression
- No Unary Operators are Present
- No Exponentiation Operators are Present
- Operands are Single Uppercase Letters That Represent Integer Values

# Applications of the Stack

## Evaluating Postfix Expressions

```
for ( each character ch in the String )
{ if ( ch is an operand)
    Push value that operand ch represents onto stack
else //ch is an operator named op
    { //Evaluate and push the Result
      operand2 = top of Stack
      Pop the Stack
      operand1 = top of Stack
      Pop the Stack
      result = operand1 op operand2
      Push result onto stack
    } // end if
} //end for
```

# Pointer-Based Implementation

```
typedef desired-type-of-stack-item StackItemType;
class Stack
{
public:
// constructors and destructor:
    Stack( );                // default constructor
    Stack(const Stack& aStack); // copy constructor
    ~Stack( );               // destructor
// stack operations:
    bool isEmpty( ) const;
    void push(StackItemType newItem);
    void pop( );
    void pop(StackItemType& stackTop);
    void getTop(StackItemType& stackTop) const;
```

# Pointer-Based Implementation

private:

```
    struct StackNode
    {
        StackItemType item; //a data item on the stack
        StackNode *next;    // points to top of stack
    }; // end struct

    StackNode *topPtr;      //pointer to first node in the stack
}; //end Stack class

// End of header file.
```

# Pointer-Based Implementation

```
Stack::Stack( ) : topPtr( NULL)
{
} // end default constructor
Stack::Stack(const Stack& aStack)
{
    if (aStack.topPtr == NULL)
        topPtr = NULL; // original list is empty
    else
    { // copy first node
        topPtr = new StackNode;
        assert (topPtr != NULL);
        topPtr->item = aStack.topPtr->item;
```

# Pointer-Based Implementation

```
// copy rest of list
    StackNode *newPtr = topPtr; // new list pointer
    for (StackNode *origPtr = aStack.topPtr->next;
        origPtr != NULL;
        origPtr = origPtr->next)
    { newPtr->next = new StackNode;
      assert(newPtr->next != NULL);
      newPtr = newPtr->next;
      newPtr->item = origPtr->item;
    } // end for

    newPtr->next = NULL;
} // end if
} // end copy constructor
```

# Pointer-Based Implementation

```
Stack::~~Stack()  
{  
    // pop until stack is empty  
    while ( !isEmpty( ) )  
        pop( );  
    // Assertion: topPtr == NULL  
} // end destructor
```

```
bool Stack::isEmpty() const  
{  
    return topPtr == NULL;  
} // end isEmpty
```

# Pointer-Based Implementation

```
void Stack::push(StackItemType newItem)
{
    // create a new node
    StackNode *newPtr = new StackNode;
    if (newPtr == NULL) // check allocation
        cout<<endl<<"stack push cannot allocate memory";
    else
    { // allocation successful; set data portion of new node
        newPtr->item = newItem;
        // insert the new node
        newPtr->next = topPtr;
        topPtr = newPtr;
    } // end if
} // end push
```

# Pointer-Based Implementation

```
void Stack::pop( )
{
    if ( isEmpty ( ) )
        cout<<endl<<"Stack empty on pop";
    else
    { // stack is not empty; delete top
        StackNode *temp = topPtr;
        topPtr = topPtr->next;
        // return deleted node to system
        temp->next = NULL; // safeguard
        delete temp;
    } // end if
} // end pop
```

# Pointer-Based Implementation

```
void Stack::pop( StackItemType& stackTop)
{
    if ( isEmpty ( ) )
        cout<<endl<<"Stack empty on pop";
    else
    { // stack is not empty; retrieve and delete top
        stackTop = topPtr -> item;
        StackNode *temp = topPtr;
        topPtr = topPtr -> next;

        // return deleted node to system
        temp->next = NULL; // safeguard
        delete temp;
    } // end if
} // end pop
```

# Pointer-Based Implementation

```
void Stack::getTop(StackItemType& stackTop) const
{
    if( isEmpty( ) )
        cout<<endl<<"Stack empty on getTop";
    else
        //stack is not empty;retrieve top
        stackTop = topPtr->item;
} // end getTop
```

```
// End of implementation file.
```